Lappeenrannan teknillinen yliopisto
School of Business and Management

Sofware Development Skills

<Israt Farabi Orpi>, <002465533>

LEARNING DIARY, < Full-Stack 2025-26> MODULE

LEARNING DIARY

13.05.2026

I started this module by going through the Full-Stack module instructions and checking what was expected from the course. I understood that the main purpose of this module is to learn how to build a fullstack application using NodeJS, MongoDB, ExpressJS, React, and the MERN stack.

For my project, I decided to build a simple Student Task Manager application. I chose this idea because it is simple but still includes the most important fullstack features. The application allows a user to add tasks, view tasks, mark them as completed, and delete them. I thought this would be a good project because it clearly shows how the frontend, backend, and database work together.

I created my main course folder in VS Code and organized it with separate files for the README, learning diary, video link, coursework, and the actual project. Inside the project folder, I created separate folders for the backend and frontend. At first, I made a few mistakes with the terminal and folder creation, but after fixing them I understood the project structure better.

Today I learned that having a clear folder structure is very important, especially in a fullstack project. It makes the project easier to manage and easier for someone else to understand.

After preparing the folder structure, I started setting up the backend. The first problem I faced was that the node and npm commands did not work in the terminal. The terminal showed that they were not recognized. I realized that Node.js was not installed correctly on my computer.

I installed Node.js and then checked the installation using:

node -v

npm -v

After this, both commands showed version numbers, so I knew the installation was successful.

Then I went to the backend folder and initialized the backend using:

npm init -y

After that, I installed the backend packages:
npm install express mongoose cors dotenv
npm install --save-dev nodemon
During this step, I learned what each package is used for. Express is used to create the backend server and API routes. Mongoose helps connect the backend to MongoDB. CORS allows the frontend and backend to communicate with each other. Dotenv is used for environment variables, and nodemon makes development easier by restarting the server automatically when changes are made.
This step helped me understand that the backend is not just one file. It needs different tools and packages to work properly.

14.05.2026
Today I worked on the backend code. I created the main backend files:

1. server.js
   .env
   models/Task.js
   routes/taskRoutes.js

In server.js, I created the Express server and connected it to MongoDB. In the .env file, I added the local MongoDB connection string and the port number. In Task.js, I created the task model using Mongoose. The task model includes the task title, description, completed status, and timestamps.
In taskRoutes.js, I created the main API routes for the project:

2. GET /api/tasks
   POST /api/tasks
   PUT /api/tasks/:id
   DELETE /api/tasks/:id

These routes allow the app to get tasks, add a new task, update a task, and delete a task. I learned that these actions are called CRUD operations, which means Create, Read, Update, and Delete.
When I first tried to run the backend, I got a MongoDB connection error. The error happened because MongoDB was not running on my computer. To fix it, I installed

MongoDB Community Server and selected the option to run MongoDB as a service. After that, I ran the backend again and it connected successfully.
When I saw:
MongoDB connected successfully
Server is running on port 5000
I felt more confident because the backend was finally working.

After finishing the backend setup, I started working on the frontend. I created the React frontend using Vite with this command:
npm create vite@latest frontend
I selected React and JavaScript as the options. Then I installed the frontend dependencies and started the frontend using:
npm install
npm run dev
The frontend opened in the browser at:
http://localhost:5173
At first, it showed the default Vite page. This was useful because it confirmed that the frontend setup was working.
I learned that in a fullstack project, the backend and frontend usually run separately. In my project, the backend runs on port 5000, and the frontend runs on port 5173. This helped me understand why both terminals need to stay open while testing the project.

15.05.2026
Today I replaced the default React page with my own Student Task Manager interface. I edited the main frontend files:

3. main.jsx
   App.jsx
   App.css

In the frontend, I created a form where the user can write a task title and description. I also created a task list where the saved tasks are displayed. The user can mark a task as completed, undo it, or delete it.

I used useState to manage changing data in the page, such as the task title, description, message, and task list. I used useEffect to load tasks automatically when the page opens. I also used fetch() to send requests from the React frontend to the Express backend. This helped me understand how the frontend and backend communicate in a real application. At this point, I could clearly see how the full MERN stack was being used:

4. NodeJS runs the backend.
   ExpressJS creates the API routes.
   MongoDB stores the task data.
   React creates the user interface.
   MERN stack combines all of these into one fullstack application.

15.05.2026

I tested the complete project by running both the backend and frontend at the same time. I kept one terminal open for the backend and another terminal open for the frontend.
I tested the main features of the project:

1. Adding a task
2. Viewing the task
3. Marking the task as completed
4. Undoing the completed task
5. Deleting the task

The project worked correctly, and the tasks were saved through the backend and MongoDB. This showed me that the frontend, backend, and database were connected properly.
After testing the project, I worked on the documentation. I updated the README file with the project description, technologies used, folder structure, setup instructions, MongoDB information, and API routes. I also created the video link file, where I will add the link to the project demonstration video.
For the video, I planned to show the backend running, the frontend running, and the main features of the application.